

DNS & Firewall Troubleshooting Report — Lab Environment (Final Structured)

1. Objective

Build a multi-server lab environment with:

- Centralized DNS (dnsmasq)
- Internal hostname resolution (*.lan)
- Controlled firewall behavior (UFW/iptables)
- Multi-tier communication (Load Balancer → App → Database)

2. Firewall Troubleshooting

2.1 Issue

Symptom:

```
ufw deny from 192.168.100.0/24
```

Ping (ICMP) traffic was still allowed from the same subnet.

2.2 Root Cause

- UFW allows ICMP by default via `/etc/ufw/before.rules`
- ICMP ACCEPT rules were placed **before** user-defined rules
- iptables processes rules **top-down (first match wins)**

Example:

```
ACCEPT icmp -- 0.0.0.0/0 icmptype 8
```

```
DROP icmp -- 192.168.100.0/24 icmptype 8
```

☞ Traffic matched ACCEPT first, so DROP never applied.

2.3 Solution

Immediate Fix (runtime)

```
sudo iptables -I INPUT 1 -p icmp --icmp-type echo-request -s 192.168.100.0/24 -j DROP
```

Persistent Fix (UFW)

Edit:

```
sudo nano /etc/ufw/before.rules
```

Add BEFORE existing ICMP ACCEPT rules:

```
-A ufw-before-input -p icmp --icmp-type echo-request -s 192.168.100.0/24 -j DROP
```

Reload:

```
sudo ufw reload
```

2.4 Validation

```
sudo iptables -L -v -n | grep icmp
```

Expected:

- DROP rule above ACCEPT
- Packet counters increasing on DROP

2.5 Key Learning

Firewall behavior depends on **rule order and processing chain**, not just rule definition.

3. DNS Troubleshooting

3.1 Issues

Issue 1 — dnsmasq failed to start

```
failed to create listening socket for port 53: Address already in use
```

Issue 2 — DNS resolution failure

```
ping app.lan → Name or service not known
```

Issue 3 — Missing tools

```
dig / nslookup not found
```

3.2 Root Causes

Port Conflict

- systemd-resolved was using port 53 (127.0.0.53)

Resolver Misconfiguration

- /etc/resolv.conf pointed to external DNS (8.8.8.8)
- dnsmasq was not being used

Minimal System

- DNS tools not installed

3.3 Solutions

Step 1 — Install tools

```
sudo apt install dnsutils -y
```

Step 2 — Disable systemd-resolved

```
sudo systemctl disable systemd-resolved
sudo systemctl stop systemd-resolved
```

Fix resolver:

```
sudo rm /etc/resolv.conf
echo "nameserver 192.168.100.102" | sudo tee /etc/resolv.conf
```

Step 3 — Configure dnsmasq (on App Server)

```
sudo nano /etc/dnsmasq.conf
```

```
#
=== DNSMASQ LAB CONFIG ===

domain-needed
bogus-priv

server=8.8.8.8
server=1.1.1.1

local=/lan/

address=/lb.lan/192.168.100.101
address=/app.lan/192.168.100.102
address=/db.lan/192.168.100.103

listen-address=127.0.0.1
listen-address=192.168.100.102

local-ttl=30
log-queries
```

Step 4 — Restart dnsmasq

```
sudo systemctl restart dnsmasq
```

Step 5 — Configure all servers

On Load Balancer and DB Server:

```
echo "nameserver 192.168.100.102" | sudo tee /etc/resolv.conf
```

3.4 Validation

```
dig app.lan @127.0.0.1
```

```
ping app.lan
```

```
ping db.lan
```

```
ping lb.lan
```

Expected:

- All hostnames resolve correctly
- Inter-server communication works

3.5 Key Learning

DNS only works when **clients actually use the intended resolver**

4. Final Architecture

Load Balancer → 192.168.100.101

App Server (DNS) → 192.168.100.102

DB Server → 192.168.100.103

All nodes use:

```
nameserver 192.168.100.102
```

5. Outcome

- Centralized DNS via dnsmasq successfully implemented
- Hostname-based communication (*.lan) operational
- Firewall rules correctly enforced
- Clear understanding of packet flow and DNS resolution

6. Final Insight

This lab demonstrated:

Firewall behavior, DNS resolution, and system configuration must align to produce expected network behavior

You progressed from:

- Executing commands to Understanding system-level interactions across networking layers